

Tool Calling in LangChain

1. Background & Motivation

- **LLM powers:**
 1. **Reasoning** → understands and breaks down queries.
 2. **Output Generation** → produces text answers.
 - **Limitation of LLMs:**
 - Cannot perform **real-world tasks** (DB update, API call, social media post, fetch live data).
 - Think of them as humans who can **think & speak** but have **no hands/legs**.
 - **Solution → Tools**
 - Functions (built-in or custom) that perform tasks.
 - LLMs can use these via **Tool Binding + Tool Calling**.
-

2. Tool Workflow

The **4-step cycle** for making LLMs actually useful:

1. **Tool Creation** → Define tool (name, description, input schema, function).
 2. **Tool Binding** → Register tool with LLM (bind_tools).
 3. **Tool Calling** → LLM decides during conversation if a tool is needed, suggests tool + input args.
 4. **Tool Execution** → Developer (via LangChain) runs tool with given inputs, returns result.
-

3. Step-by-Step Concepts

◆ Tool Creation

- Tools are Python functions with extra metadata.

```
from langchain.tools import tool
```

```
@tool
```

```
def multiply(a: int, b: int) -> int:
```

```
    """Given two numbers a and b, returns their product."""
```

```
    return a * b
```

- Each tool must define:
 - **Name**
 - **Description** (helps LLM know what it does)
 - **Input schema** (expected args & types)
-

◆ Tool Binding

- Connect tool(s) with an LLM.

```
from langchain_openai import ChatOpenAI
```

```
llm = ChatOpenAI(model="gpt-4o-mini")
```

```
llm_with_tools = llm.bind_tools([multiply])
```

- After binding, LLM knows:
 1. Which tools exist.
 2. What each tool does.
 3. What input format they expect.
-

◆ Tool Calling

- LLM **suggests** a tool call when needed.

- Output looks like:

```
{  
  "tool": "multiply",  
  "arguments": {"a": 8, "b": 7}  
}
```

⚠ **Important:** LLM does not execute the tool itself → only suggests!
Execution is handled by **LangChain** + **programmer**.

◆ Tool Execution

- Programmer executes suggested tool with inputs:

```
result = multiply.invoke({"a": 8, "b": 7})
```

- Returns output (e.g., 56).
 - Wrapped in a **ToolMessage** so LLM can continue conversation with updated context.
-

4. Example – Currency Conversion App

Problem:

- LLMs have outdated knowledge → can't give real-time conversion (USD → INR).

Solution:

- Build 2 tools:
 1. **get_conversion_factor** → hits an API to fetch live rate.
 2. **convert** → multiplies amount × rate.

Flow:

1. User asks: “Convert 10 USD to INR.”
2. LLM suggests:
 - Call `get_conversion_factor("USD", "INR")`
 - Then call `convert(10, 85.34)`
3. Programmer executes both sequentially → returns final answer.

Handling Dependencies → Injected Tool Arguments

- Issue: LLM might guess wrong conversion rate from training data.
- Fix: Use **InjectedToolArg** so programmer injects latest value.

```
from langchain_core.utils.function_calling import InjectedToolArg  
from typing import Annotated
```

```
@tool
```

```
def convert(base_value: int, conversion_rate: Annotated[float, InjectedToolArg]) -> float:  
    """Converts base_value using provided conversion_rate."""  
    return base_value * conversion_rate
```

5. Key Insights

- **Tool Binding** → tells LLM what tools are available.
 - **Tool Calling** → LLM suggests tool usage, not execution.
 - **Tool Execution** → controlled by programmer, safer & reliable.
 - **Injected Arguments** → let programmer feed dynamic values (avoid LLM hallucination).
 - **Applications:**
 - Real-time data fetching (weather, currency, stock).
 - Automating workflows (emails, DB queries).
 - Building AI agents (foundation step).
-

6. Interview Q&A

Q1. Why can't LLMs perform tasks directly?

- Because they only generate text; they lack environment access. Tools extend this capability.

Q2. Difference between Tool Calling & Tool Execution?

- Tool Calling → LLM **suggests** which tool to use + args.
- Tool Execution → Developer (via LangChain) actually runs tool.

Q3. Why is Tool Binding important?

- Without binding, LLM doesn't know tools exist or how to format input.

Q4. What problem do Injected Tool Arguments solve?

- Prevent LLM from hallucinating values for arguments that must come from previous tool outputs.

Q5. Is this an AI Agent?

- No. True agents = autonomous. Here programmer controls execution. Agents will handle everything automatically.